

```

import streamlit as st
import yfinance as yf
import pandas as pd
import numpy as np
from datetime import datetime, time, timedelta
import pytz

st.set_page_config(page_title="株価分析&類似チャート照合ツール", layout="centered")

st.title("📈 株価分析&類似チャート照合アプリ")
st.write("銘柄コードを入力して、昨今の値動きと過去の類似パターンを分析します。")

# 1. ユーザー入力
ticker_symbol = st.text_input("銘柄コード (例: 7203.T, AAPL) ", value="7203.T")

# 2. 時間判定ロジック (JST基準)
jst = pytz.timezone('Asia/Tokyo')
now_jst = datetime.now(jst)
current_time = now_jst.time()

# 15:30を境界線とする
threshold_time = time(15, 30)
if current_time >= threshold_time:
    target_date_type = "今日 (最新) のチャート"
else:
    target_date_type = "昨日のチャート"

st.info(f"現在時刻: {now_jst.strftime('%H:%M')} -> 基準判定: **{target_date_type}**")

if st.button("分析を実行する"):
    with st.spinner("株価データを取得し、類似チャートを検索中..."):
        try:
            # データの取得 (過去3年分)
            stock = yf.Ticker(ticker_symbol)
            df = stock.history(period="3y")

            if df.empty:
                st.error("データを取得できませんでした。銘柄コードを確認してください。")
            else:
                # 直近のデータ表示
                latest = df.iloc[-1]
                prev = df.iloc[-2]

                st.subheader("📊 価格変動と出来高")

```

```

coll1, col2 = st.columns(2)
with coll1:
    st.metric("終値", f"{latest['Close']:.2f}", f"{latest['Close']}")
    st.metric("始値", f"{latest['Open']:.2f}")
with col2:
    st.metric("高値", f"{latest['High']:.2f}")
    st.metric("安値", f"{latest['Low']:.2f}")

vol_change = ((latest['Volume'] - prev['Volume']) / prev['Volume'])
st.metric("出来高前日比", f"{latest['Volume']:,} 株", f"{vol_change:.2f}%")

# トrend判定の簡易ロジック (20日移動平均線と比較)
df['SMA20'] = df['Close'].rolling(window=20).mean()
current_close = latest['Close']
current_sma = df['SMA20'].iloc[-1]

if current_close > current_sma * 1.02:
    trend = "上昇トレンド"
elif current_close < current_sma * 0.98:
    trend = "下降トレンド"
else:
    trend = "レンジ相場"

st.write(f"***トレンド判定:** {trend}")

# 類似チャート検索のロジック (直近5日間の変動率パターンで比較)
st.subheader("🔍 過去の類似チャート照合 (直近5日間の値動き)")

# 直近5日間のリターン計算
recent_window = 5
if len(df) > recent_window + 300:
    recent_returns = df['Close'].pct_change().iloc[-recent_window:]

# スライディングウィンドウで過去の類似度 (相関係数) を計算
best_corr = -1
best_idx = None

close_series = df['Close']
for i in range(recent_window, len(df) - recent_window - 1):
    past_returns = close_series.pct_change().iloc[i-recent_window:i]
    if len(past_returns) == recent_window and not np.isnan(past_returns).all():
        # 相関係数を計算
        corr = np.corrcoef(recent_returns, past_returns)[0, 1]
        if not np.isnan(corr) and corr > best_corr:
            best_corr = corr
            best_idx = i

```

```

if best_idx is not None and best_corr >= 0.8:
    matched_date = df.index[best_idx].strftime('%Y-%m-%d')
    next_day_return = (df['Close'].iloc[best_idx + 1] - df['C

    st.success(f"一致度 {best_corr*100:.1f}% の類似パターンを発見！")
    st.write(f"- **類似した過去の日付:** {matched_date}")
    st.write(f"- **その翌日の値動き:** {'📈 上昇' if next_day_re
else:
    st.info("一致度80%以上の類似チャートパターンは直近データからは見つ
else:
    st.warning("データ量が不足しているため、類似チャートの照合をスキップしま

except Exception as e:
    st.error(f"エラーが発生しました: {e}")

```